

CPT202 Software Engineering Group Project

Final Report for Software Engineering Group Project

2025/2026 Semester 2

Consilium - a booking consultation system

Group number: 22

Student Name: Qi Cao

Student ID: 2360908

<http://121.41.36.107/>

Admin account: username = admin@gmail.com, password = Password123

Specialist account: username = Priya.Sharma@gmail.com, password = Password123

Customer account: username = Alice@gmail.com, password = Password123

2026.5.30

Contents

1. Introduction.....	3
2. Software Development Process	3
3. Software Design.....	6
4. Change Management	10
5. Legal, social, ethical, and professional issues.....	11
6. Conclusion	12
References	13
Appendix	14

1. Introduction

In recent years, with the development of internet, more and more people prefer to make consultation online rather than spending extra time and effort arranging consultations offline. However, the current web-based booking systems for consultation still lack sufficient efficiency, flexibility, and reliability in managing the booking process. These limitations may lead to inconvenient user experiences, inconsistent schedule management, and difficulties for staff in handling bookings, specialist information, and pricing rules. To solve these problems, we developed a web-based booking system for a consultancy service provider. The system supports three types of key stakeholders: customers, specialists and administrator. Customers can search for experts through professional fields and available time, and submit appointment requests. Specialists can edit personal information and process appointment requests. Administrators can review specialist applications, manage user accounts and monitor all appointment records.

As a developer in our team, I was mainly responsible for five Product Backlog Items (PBIs) related to Administrator module. My work focuses on realizing the core functions of administrators, including specialist review, user management, booking monitoring and service announcement management. I developed the relevant back-end API and connected it with the front-end page. At the same time, I make sure that only authorized administrator can access these management functions.

In my opinion, the advantage of the current version is that the system has good module clarity and role-oriented functionality. The functional processes of administrators, experts and customers are clearly separated, and users can choose specific roles according to their needs. The quality of the current version is relatively stable, because we have continuously tested it during the development process.

This report will first introduce the software development process and focus on my personal contribution to the team. Then, the software design section will outline my work and explain in detail the two key PBIs. After that, the change management part will explain how the demand changes are handled and managed in my development process. Finally, I will discuss some legal, social, ethical and professional issues related to the project, and explain how they affect development decision-making, while giving some possible solutions.

2. Software Development Process

2.1 Scrum-based Development Process and Sprint Planning

In our project, we used Scrum to manage the software development process. Scrum is an agile method that divides the whole development process into several short and manageable iterations called sprints. This approach allowed our team to break down the system into smaller Product Backlog Items (PBIs), assign tasks clearly, and review development progress regularly[1]. Our development lifecycle was divided into fixed-length, two-week Sprints. This helped us plan tasks, track progress, test completed

features and make improvements based on team discussion and feedback. Specifically, as Fig. 1 shown, we divided the whole tasks into three sprints, namely Foundation and UI Framework, Role-Based Core Module Development, Finalization, Testing and Deployment.

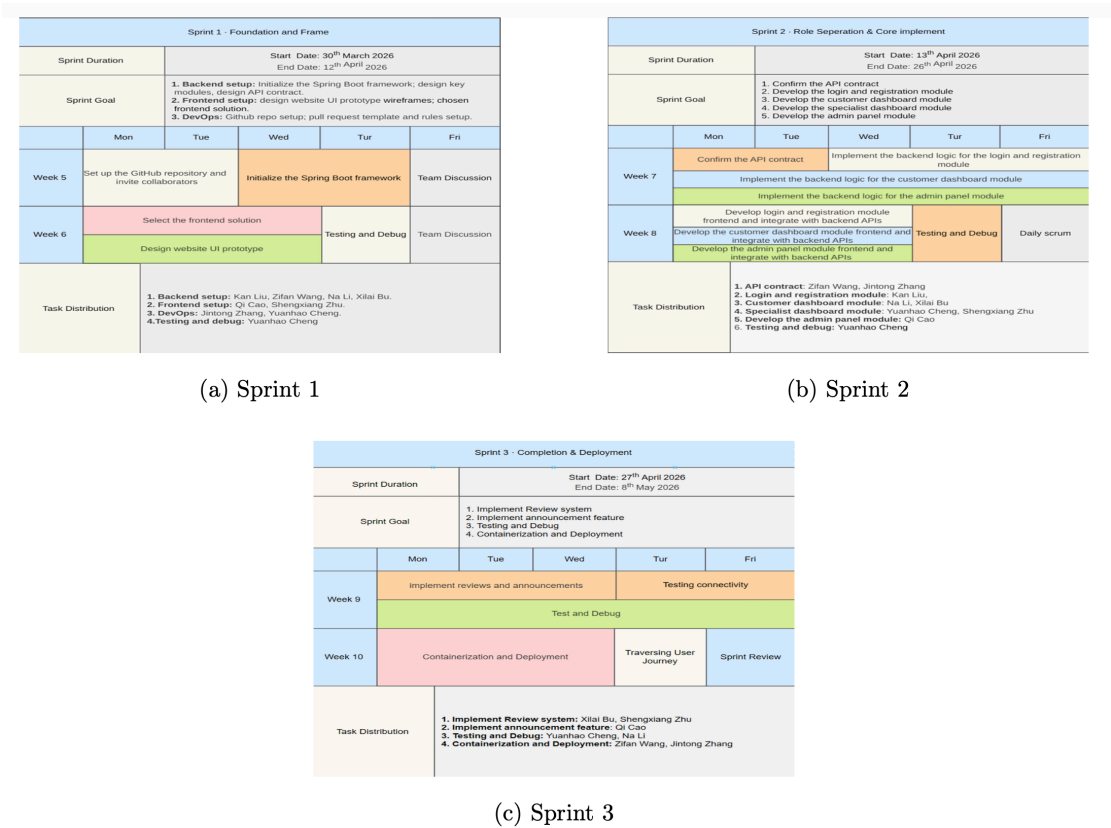


Fig.1. Sprint Backlogs

In Sprint 1, our team focused on the early preparation for system development. We discussed the overall system requirements, designed the initial database structure (Fig.1 in Appendix), and created the UI prototype. During this sprint, I mainly contributed to the design of the announcement-related and Categories database tables. I designed the first version of the admin panel UI (layout + main sections). This served as the basis for the core admin module in Sprint 2.

In Sprint 2, our main task was to implement the core system functions. I developed the specialist review, user management and booking monitoring functions of the Administrator module. I developed the related backend logic and APIs and tested them locally to ensure that they returned the expected results and supported the required frontend operations. These functions allowed administrator to review specialist applications, manage user accounts, and monitor booking records across the system. This process helped ensure that the Administrator module could work properly with the rest of the project.

In Sprint 3, our team focused on completion functions, testing, and deploying. In this Sprint, main task for me was to implement the announcement feature in the Administrator module (Fig.2 in Appendix). This feature was implemented in the last

sprint because it was relatively independent from other modules and did not require close coordination with other modules' developers. I implemented the functions for creating, updating, and deleting service announcements, and connected them with the administrator interface. After completing this feature, I started testing of other functions of the system, checked API and debugged several issues found during integration testing.

2.2 Communication and Coordination

Communication and coordination were important in my development because the Administrator module depends on many other modules in the system (Fig.3 in Appendix). For example, monitoring booking records function in my module is depended on the "Booking Appointment Flow" module and the "Booking Status and Lifecycle" module because administrator needs to check booking details, fees and current booking statuses. To correctly implement this function, I held an online meeting with the team members responsible for these parts. We confirmed the API dependencies and integration requirements and discussed the `/api/admin/bookings` response format — including reservation ID, customer info, expert info, consultation time, fee, and reservation status. This coordination reduced integration risk and helped us deliver a consistent cross-module experience.

2.3 Engineering Practices and Tools

We used GitHub for codes management and version control. In Sprint 1, we created a GitHub repository and invited all our members. Each member has their own branches (Fig.4 in Appendix). During the development, I first wrote and tested codes in my own development environment, then I committed my codes to my own GitHub branch and created a pull request for team review. After my branch was merged into the main branch, I worked with other team members to integrate my functions with the overall system and resolve any conflicts or integration issues. During the testing stage, when I found bugs or functional issues, I fixed them and tested the updated function again before committing the changes. For example, I found that the specialists' profile form allowed invalid fee inputs, such as zero or negative values. I fixed this bug by adding validation to ensure that the fee must be greater than 0 before submission. After testing the fix, I committed the change to GitHub with a clear commit message (Fig.5 in Appendix), which made this issue and its solution traceable in the project history.

2.4 Challenges and Solutions

One of the main challenges I encountered in the development process is how to ensure that my functions have passed enough real-life scenario tests. When the administrator module was initially implemented, I mainly focused on whether the normal process could run successfully, such as viewing users, managing classifications, creating announcements, etc. However, in the subsequent tests and discussions, I realized that the administrator may also face many abnormal situations. For example, the administrator may try to reject the expert's application without providing a reason for rejection, or create an announcement of missing content. In order to solve this challenge,

I added a check for invalid operations, tested both success and failure scenarios, and ensured that the system can return clear feedback when the operation cannot be completed. This helps to improve the reliability of the administrator module.

3. Software Design

3.1 Overview of My Developed PBIs

In this project, I was mainly responsible for four Product Backlog Items (PBIs) related to Administrator module. They are PBI 1. Booked Records Monitoring, PBI 2. System Announcements Management, PBI 3. User Management and PBI 4. Expertise Categories Management. Their detailed information is shown in Fig.6 in Appendix.

3.2 Detailed Discussion of Two Essential PBIs

In this section, I will give a detailed discussion of two important PBIs: PBI 2. System Announcements Management and PBI 4. Expertise Categories Management.

3.2.1 PBI2.System Announcements Management

(1). Brief description and Operational context

This PBI allows administrators to create, view, edit, publish or unpublish, and delete system announcements through the administrator panel. In the operational context of a consultancy booking system, administrators may need to inform users about important service updates, platform maintenance and other operational notices. Without this function, the system would lack a centralized way to deliver important information to users. Therefore, this PBI helps administrators communicate service information more efficiently and ensures that users can receive important notices directly through the platform.

(2). Logical flow

The logical flow of this PBI is shown in the following Sequence Diagram (Fig.2). First, the administrator logs into the administrator panel and accesses the page to manage system announcements. The frontend requests all existing announcements and the backend verifies admin authorization and then returns the full list for display. To create a new announcement, the administrator fills in the title, body, target audience, etc. of the announcement and the frontend system submits the entered data to the backend system. In the backend system, the entered data is validated and the new announcement is stored in the database. Then the result of creation of the new announcement is returned. For the maintenance of existing announcements, the administrator changes the information of announcement and uses PUT /api/admin/announcements/{id} to updates the record of the announcement that is to be maintained in the database and returns the latest state of the announcement. For deletion of an announcement that is no longer needed, the administrator clicks the button for deletion of the announcement. Then the frontend system calls DELETE /api/admin/announcements/{id} and the backend system removes the announcement from the database that was announced to be deleted by the administrator and the system confirms the successful deletion of the announcement.

(3). Database support

The database in Fig. 3 supports the PBI through the tables ANNOUNCEMENTS, USERS and ANNOUNCEMENT_READS. The ANNOUNCEMENTS table is used to store the announcements' data such as title, body and timestamps. The ANNOUNCEMENT_READS table is used to link users with announcements. Each read announcement of a user is recorded in the read_at time. Therefore, the system can verify whether a user has read a specific announcement or not. This structure of database records the announcements' content and users' read status separately, thus the system is clear and can be extended in the future.

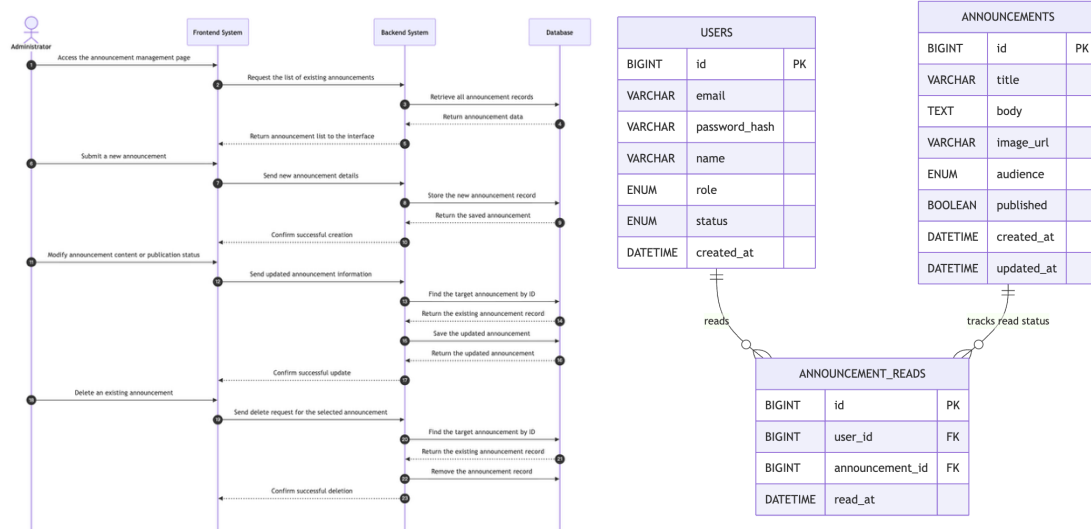


Fig.2. The Sequence Diagram of Announcements Management

Fig.3. The ERD of Announcements Management

(4). Object states and behavior changes

In this PBI, the main object is Announcement. An announcement can be unpublished, published, updated, or deleted. When an administrator creates an announcement, it is stored in the database. If it is unpublished, users cannot see it. When the administrator publishes it, the announcement becomes visible to the selected audience. If the administrator updates the announcement content or publication status, the system saves the latest changes and updates the updated_at field. Then the selected audience will see the new updated announcement. If the announcement is deleted, it is removed from the system.

(5). UI design support

The UI design for this PBI is mainly implemented on the announcements management page of the administrator panel, as shown in Figure 7 in Appendix. There is a list that displays all the existing announcements, and each announcement in the list includes its information and several buttons for administrator's operations. The Unpublish, Edit and Delete buttons are provided for administrators to change an announcement's publication status, to edit the announcement's content, and to delete the announcement. Note that the Delete button is highlighted in red to alert administrators that they are going to delete an announcement before they confirm the deletion. The New announcement button is provided so that administrators can create new announcements

by filling in a new announcement's information.

(6). Design decisions and trade-offs

In the earlier design, announcement management could be implemented as a simple CRUD feature. However, after considering user interaction, we added the ANNOUNCEMENT_READS table to record whether each user has read a specific announcement. This design separates announcement content from user-specific read status, making the system clearer and more extensible. The trade-off is that the database design becomes more complex and requires additional logic to create or update the read records.

(7) Validation

To check out the functionality of PBI 2 after its implementation, first the unit test for the isolated object or function is run. Then the integrated test for the frontend user interface combined with the backend APIs, and the database storing the data are conducted. The test results are represented in Figure 8 in Appendix.

3.2.2 PBI4. Expertise Categories Management

(1). Brief description and Operational context

This PBI is used by administrator to add, edit, delete categories. In the operational context of a consultancy booking system, these categories are used to group the organization's specialists by their areas of expertise, so that customers can more easily search and filter through the different specialists available for booking. The administrator can use these categories to organize and manage the information of the specialists in order to make it easier for customers to book the right experts for their projects.

(2). Logical flow

The process to manage expertise categories through the admin panel is explained in Fig. 4. First, administrator opens the page for managing Categories. The frontend system sends a request to GET /api/admin/categories to get all existing categories from the backend and displayed them in a management table. Categories can be added by entering the name of new category in the form provided by the interface. The newly added category will be stored in the database by backend after validating the POST /api/admin/categories request. For maintenance, the administrator can edit an existing category name and description through PUT /api/admin/categories/{id}, and the backend updates the existing record in database and returns the latest category data. If a category is no longer needed, the administrator clicks the deletion button. Then the backend checks whether any specialist is currently using it. If the category is in use by any specialist, then an error message is displayed and category will not be deleted. If no specialist is using the category to be deleted, then the category is deleted from the database by the backend and confirms the successful deletion.

(3). Database support

Fig. 5 shows the ER diagram of Expertise Categories Management. I used a CATEGORIES table and a SPECIALISTS table to implement this PBI. The CATEGORIES table stores all categories of expertise managed by an administrator. This table allows the administrator to add, update or delete a category of expertise. The SPECIALISTS table was related to the CATEGORIES table through the category_id field of the table. This allows for the classification of specialists and searching and filtering of specialists by a particular area of expertise.

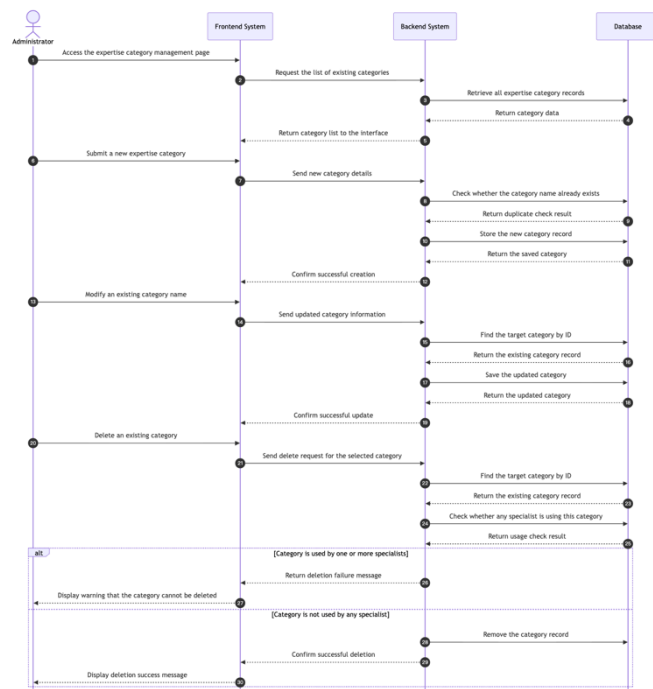


Fig.4. The Sequence Diagram of Expertise Categories Management

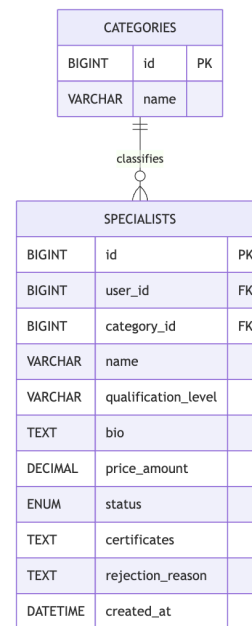


Fig.5. The ERD of Expertise Categories Management

(4). Object states and behavior changes

In this PBI, the main object is Category. A category is first created by the administrator and stored in the database with its category ID and category name. After creation, the category becomes available in the system and can be used when specialists set up or update their profiles. Customers can also use these categories to search or filter specialists by expertise area. If the administrator updates the category object's name, the system saves the new category information and displays the latest category name in the management page. The related specialist profile display and search/filter results will also be updated. If the category is not used by any specialist, the administrator can delete it, and it will be removed from the category list. However, if the category is already assigned to one or more specialists, the system refuses the deletion and returns an error message.

(5). UI design support

The UI design supports this PBI through the categories management page in the Administrator panel. As shown in Fig.9 in Appendix, existing categories are displayed in a list. The Edit and Delete buttons are placed next to each categories' names, allowing the administrator to edit or delete the categories. The categories' Delete buttons are

highlighted in red to alert the user before they carry out the action to remove the categories to avoid an accidental deletion. The administrator would click on the orange highlighted Add category button and enter the name of the new categories to add a new category to the existing list.

(6). Design decisions and trade-offs

For category deletion, we chose to stop deletion when the category is already used by specialists. This design avoids damaging specialist profile data. If a category is deleted while specialists are still assigned to it, their profiles may become incomplete. The customers will have some problems when searching by categories. The trade-off is that administrator needs to first reassign or update related specialists before deleting a category. Although this adds an extra step, it provides better data protection and system reliability.

(7) Validation

After finishing the development of this PBI, I first used unit testing to check whether the core functions worked correctly in isolation. Then, I used integration testing to verify whether the frontend interface, backend APIs, and database worked together properly. The results are shown in Fig.10 in Appendix.

4. Change Management

During the development process, a requirement change related to my work was in the User Management function of the Administrator module. In the beginning, the administrator mainly expected to view user information, such as the user list and user details. However, after further discussion, the requirement changed. The administrator also needed to be able to control whether a user account could continue using the platform. Because during our testing stages, some users made bad comments and behaved improperly. Therefore, the administrator should be able to give some temporary restrictions to these users. Then, the requirement extended from simple user viewing to user status management.

This change affected PBI 3. User Management. Now it not only needed to include the function of viewing user information, but also the function of enabling and disabling user accounts. The design and implementation are also influenced. A status field needs to be added to the USERS table to indicate whether the account is allowed to use the system. Meanwhile, I needed to add backend logic and APIs for changing user status, and the frontend needed to provide operation buttons for administrators to enable or disable users. This change also required additional tests. We needed to test whether the administrator could update user status successfully, whether the updated status was displayed correctly in the Administrator panel, and whether disabled users were restricted from using the system as expected.

Since this change occurred in our development process, the team needs to adjust the original plan and task assignment. We discussed this change in the team meeting and

decided to manage it as an extension of the existing user management module instead of creating a new module. Some team members were assigned to help me implement and test the new user status management function because they had already completed their main tasks at that stage. This enabled us to handle the requirement change without delaying the overall sprint progress too much.

5. Legal, social, ethical, and professional issues

5.1 Legal Issues

The database in this system stores account information for registered users, details of the experts on this system, records of past and future appointments and other relevant details about users. All of the above information hold privacy issues that would be exposed if the information was to leak or be accessed by some unauthorized users. Hence this issue affects the requirements and the scope of this project. To solve this problem, the system should adopt a role-based access control mechanism to ensure that customers, experts and administrators can only access information related to their roles. For example, in the administrator module I developed, functions such as user management and global appointment monitoring should only be accessed by the administrator.

5.2 Social Issues

Accessibility and usability are important social issues because the system is aimed at different types of users, including customers, experts and administrators. These users have different backgrounds and technical experiences. Therefore, this problem affects our UI design decision-making. In order to solve this problem, the interface should provide clear labels, easy-to-understand error messages, and confirmation feedback after important operations (such as creating appointments, approving experts, deleting classifications). For example, in the administrator module, the deletion operation should clearly inform the administrator whether the operation is performed successfully or blocked due to dependencies.

5.3 Ethical Issues

Since our system is an online consultation booking system, customers need to compare specialists and make appointment decisions based on professional knowledge, available time, consultation fees and other information. Therefore, the fairness and transparency of pricing are important ethical issues. If the expert sets different prices based on the customer's personal information, it may lead to unfair treatment or price discrimination. This problem affects our project needs and pricing design. To overcome this problem, we set a fixed hourly consultation fee for each specialist at the beginning and this fee will be applied for all customers. This makes the pricing rules clearer and reduces the risk of unfair pricing.

5.4 Professional Issues

Since our system is developed for real customers and service providers, reliability and maintainability are very important. We have responsibility to make system useable,

stable and safe for customers and service providers. We dealt with this issue by maintaining communication among team members and carrying out detailed tests on system throughout development process. We use GitHub for version control. For example, we are developing functions on independent branches and making pull requests before we merge these with the main branch where we keep our code. In this way, we make sure that changes we made on code are traceable and reduced any accidental conflict. Thus, we make system stable and easier to use for customers. These make it possible to carry out maintenance and developments easily in the future.

6. Conclusion

Key Learnings

Through this project, I have learned about technical tools such as Spring Boot, JavaScript, MySQL and Docker. I also gained a deeper understanding of teamwork and collaboration in software development. By using the Scrum framework, our team splits the whole project into smaller and easier-to-manage tasks, which makes the development process more orderly and easier to track. I also realize that frequent communication in team projects is crucial, especially when there is interdependence between different modules. Regular discussions help us clarify the needs, solve integration problems, and keep the project consistent with the sprint goal. In general, this project has improved my technical skills and my ability to collaborate efficiently in the development team.

Reflections

In the past, I often just focused on whether my code could run correctly instead of paying attention to the code style and long-term maintainability. Through this project, I realized that a single developer is not independent. We must ensure that the code can be understood, tested, maintained and integrated with other modules. Therefore, I am more aware of the importance of module dependence, API consistency, code readability, testing and communication with other developers. The current project contributed significantly to my growing-up as a software engineer.

Future Improvements

From the technical perspective, I hope to improve my backend development skills in future projects. Although I have learned many techniques in this project, I still need to have a deeper understanding of backend architecture, security, database optimization and testing strategies. For future iterations of this project, I will add an intelligent recommendation system to recommend suitable specialists to customers based on their historical information. This can improve the user experience by helping customers find appropriate specialists more efficiently. In addition, I will use HTTPS to provide safer data transmission between the client and the server. This would help to protect sensitive information.

From the process perspective, I hope to become more familiar with Scrum and apply it better in future teamwork. In this project, we used sprints, PBIs, and daily scrum

meetings to manage development, but there is still room for improvement in sprint planning, task allocation, progress tracking, and sprint review. In future projects, I will try to define tasks more clearly at the beginning of each sprint and communicate with team members more frequently.

References

- [1] F. Hayat, A. U. Rehman, K. S. Arif, K. Wahab, and M. Abbas, “The influence of agile methodology (Scrum) on software project management,” in *Proc. 20th IEEE/ACIS Int. Conf. Software Engineering, Artificial Intelligence, Networking and Parallel/Distributed Computing (SNPD)*, 2019, pp. 145–149.

Appendix

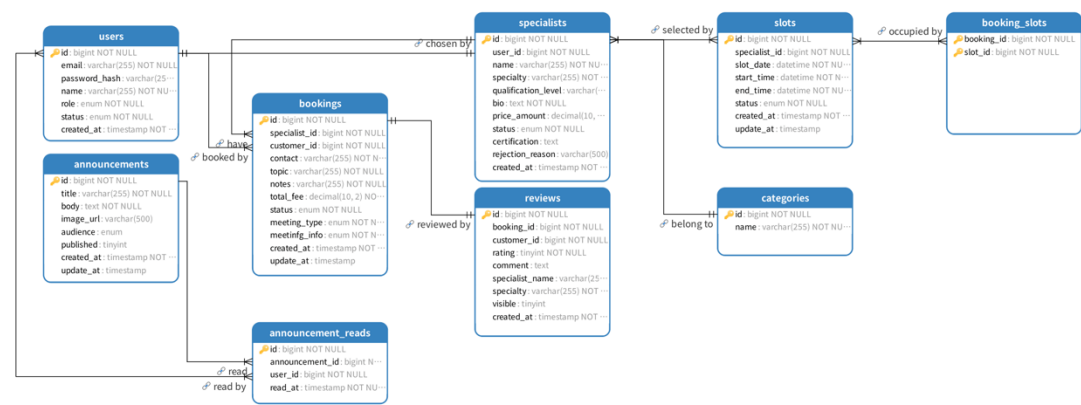


Fig.1. Overall ER Diagram

feat: add announcement function, fix sum bugs

Wonder239239 committed 3 weeks ago

Fig.2. GitHub commit

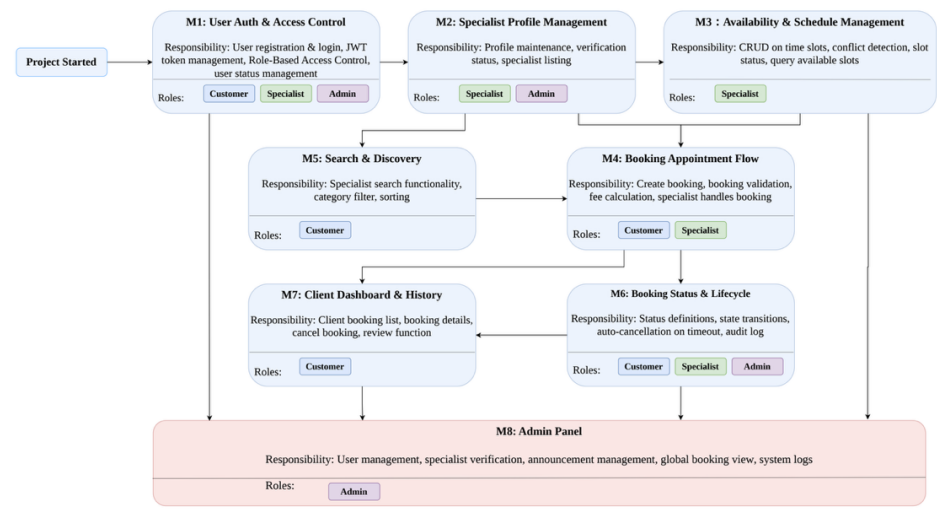


Fig.3. Module Relationship Diagram

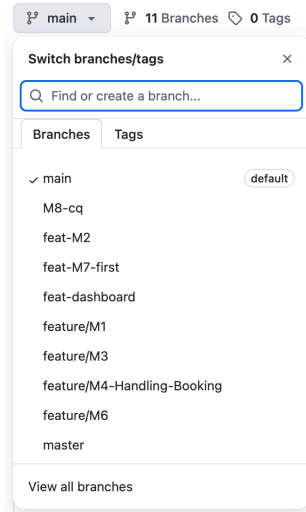


Fig.4. GitHub branches

feat: validate fee input in profile form to ensure it is greater than 0

Wonder239239 committed 3 weeks ago

Fig.5. GitHub commit of fixing bugs

3.1 PBI 1: Monitor Booking Records

3.1.1 User Story

As an administrator, I want to view and monitor all booking records so that I can ensure correct system operations.

3.1.2 Acceptance Criteria

1. Display Booking Records
 - **GIVEN** I am an administrator,
 - **WHEN** I access the booking monitoring page on the admin panel,
 - **THEN** I should see the list of booking records with customers, specialists, booking time, and booking status(Pending, Confirmed, Cancelled, Completed).
2. Filter Booking Records by Status
 - **GIVEN** I am an administrator,
 - **WHEN** I apply a status filter when viewing booking records on the admin panel,
 - **THEN** I should see only the booking records that match the selected status. If no records match the selected status, the system should display a message indicating that no results are found.

3.3 PBI 3: Manage System Announcements

3.3.1 User Story

As an administrator, I want to publish system announcements so that I can inform users of updates and other information.

3.3.2 Acceptance Criteria

1. Create System Announcement
 - **GIVEN** I am an administrator and I want to release an announcement,
 - **WHEN** I create and release a new announcement on the admin panel,
 - **THEN** all logged-in users should see the announcement on their dashboards.
2. Delete System Announcement
 - **GIVEN** I am an administrator and an announcement is no longer needed,
 - **WHEN** I delete the announcement on the admin panel,
 - **THEN** the announcement on users' dashboards disappears.
3. Update System Announcement
 - **GIVEN** I am an administrator and an announcement needs to be update,
 - **WHEN** I update the announcement on the admin panel,
 - **THEN** the original announcement is replaced by the new announcement on users' dashboards and marked as "Unread".

3.2 PBI 2: Manage User Accounts

3.2.1 User Story

As an administrator, I want to view and manage user accounts so that I can monitor system users and manage their access to the system.

3.2.2 Acceptance Criteria

1. Display User List
 - **GIVEN** I am an administrator,
 - **WHEN** I access the user accounts page on the admin panel,
 - **THEN** I should see a list of all users with their roles and account statuses.
2. View User Details
 - **GIVEN** I am an administrator and I select a user,
 - **WHEN** I open the user profile,
 - **THEN** I should see the user's detailed information, including user's name, user's identity(specialist or customer) and user's status.
3. Disable User Account
 - **GIVEN** I am an administrator and I want to disable a user account,
 - **WHEN** I click the "disable" button on the page,
 - **THEN** the user should no longer be able to access the system.
4. Enable User Account
 - **GIVEN** I am an administrator and a user account is disabled,
 - **WHEN** I click the "enable" button on the page,
 - **THEN** the user should be able to access the system again.
5. Restrict Unauthorized Access
 - **GIVEN** I am not an administrator,
 - **WHEN** I attempt to access user management features,
 - **THEN** access should be denied.

3.4 PBI 4: Manage Expertise Categories

3.4.1 User Story

As an administrator, I want to manage expertise categories so that specialists can be properly classified and users can search for services more effectively.

3.4.2 Acceptance Criteria

1. Add Expertise Category
 - **GIVEN** I am an administrator,
 - **WHEN** I create a new expertise category on the admin panel,
 - **THEN** the new category is generated and should be available for specialists to select and the user to choose.
2. Update Category Information
 - **GIVEN** I am an administrator and an existing category needs to be updated,
 - **WHEN** I modify the information of the category on the admin panel,
 - **THEN** the updated category should be reflected across the system.
3. Delete Category
 - **GIVEN** I am an administrator and a category is no longer needed,
 - **WHEN** I delete the category,
 - **THEN** the category should disappear on users' dashboards. If the category is associated with any specialists, the system should display a warning message to the administrator to prevent deletion.
4. Restrict Unauthorized Access
 - **GIVEN** I am not an administrator,
 - **WHEN** I attempt to manage categories,
 - **THEN** access should be denied.

Fig.6. PBIs

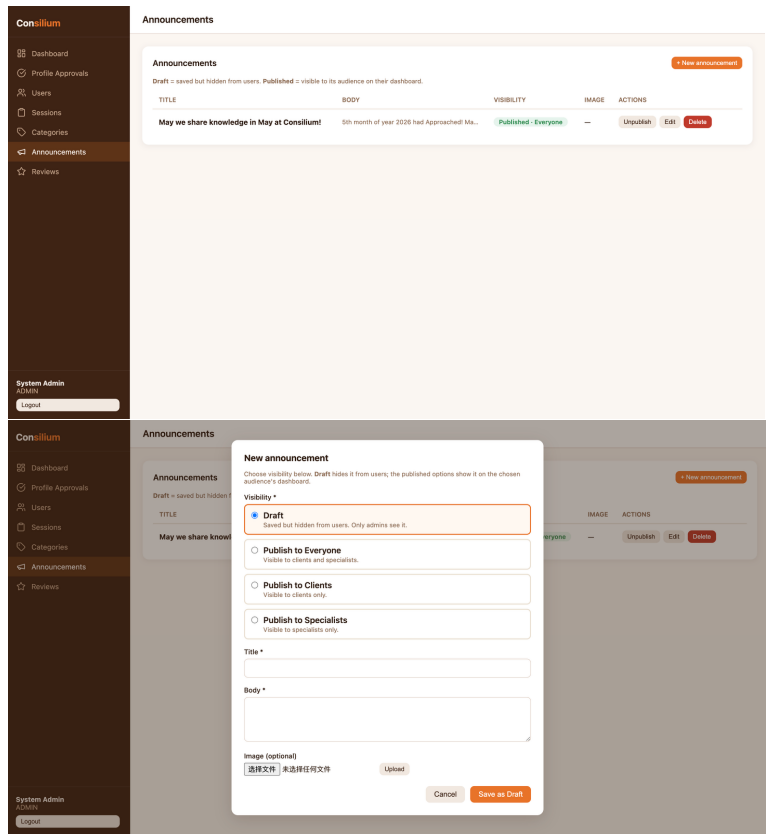


Fig.7. UI design of PBI 2 Announcement Management

✓ AnnouncementServiceTest (com.grooming.pet.service)	572毫秒
✓ markAsRead_shouldNotInsertRecord_whenAlreadyRead()	557毫秒
✓ createAnnouncement_shouldPersistPayload_whenAdmin()	5毫秒
✓ getAnnouncementsForCurrentUser_shouldFilterByAudienceAndUnreadFlag()	3毫秒
✓ markAsRead_shouldThrowNotFound_whenAnnouncementDoesNotExist()	2毫秒
✓ getAllAnnouncements_shouldRejectNonAdmin()	2毫秒
✓ markAsRead_shouldInsertRecord_whenNotPreviouslyRead()	3毫秒
✓ AnnouncementManagementIntegrationTest (com.grooming.pet.controller)	2秒 262毫秒
✓ createAnnouncement_shouldValidatePayloadAndAudience()	561毫秒
✓ getMine_shouldRequireAuthentication()	19毫秒
✓ adminEndpoints_shouldRejectNonAdminUser()	311毫秒
✓ adminCrudFlow_shouldManageAnnouncementSuccessfully()	177毫秒
✓ userFlow_shouldFilterAudienceAndMarkRead()	312毫秒
✓ updateDeleteAndRead_shouldReturnNotFoundForMissingAnnouncement()	291毫秒
✓ specialistFlow_shouldSeeAllAndSpecialistAnnouncementsOnly()	296毫秒
✓ markRead_shouldBeIdempotent_whenCalledMultipleTimes()	295毫秒

Fig.8. Testing of PBI 2 Announcement Management

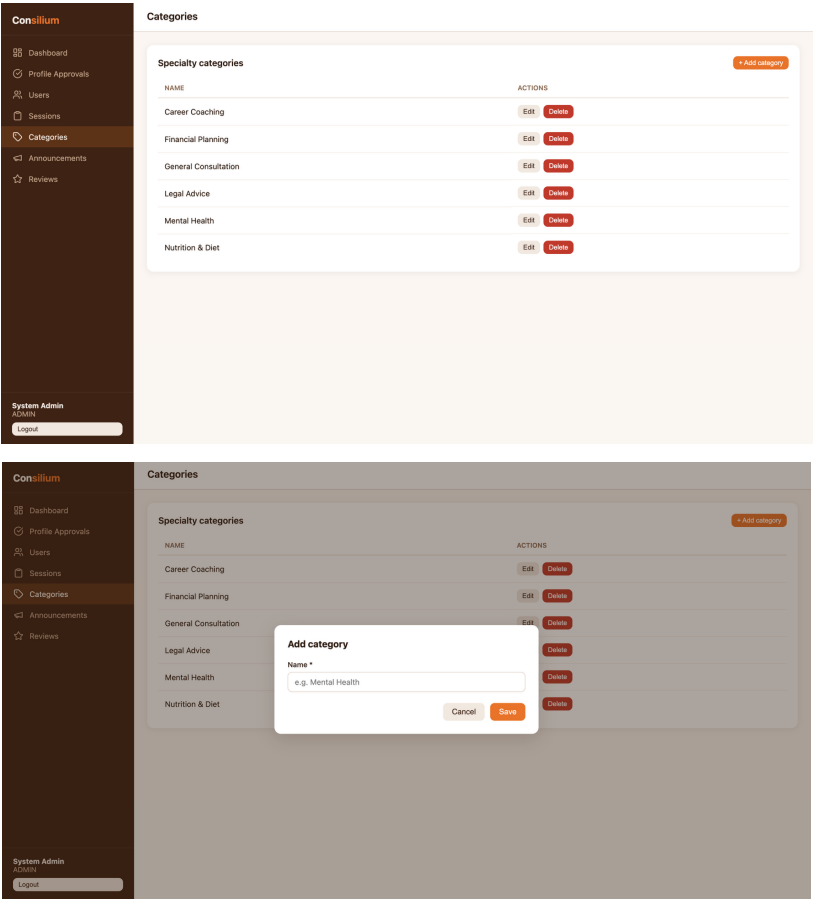


Fig.9. UI design of PBI 4 Expertise Categories Management

✓ CategoryServiceTest (com.grooming.pet.service)	427毫秒
✓ deleteCategory_shouldThrowBadRequest_whenCategoryInUse()	418毫秒
✓ createCategory_shouldPersistAndReturnResponse_whenInputValid()	4毫秒
✓ getAllCategories_shouldReturnMappedResponses()	2毫秒
✓ createCategory_shouldThrowForbidden_whenCurrentUserIsNotAdmin()	1毫秒
✓ createCategory_shouldThrowConflict_whenCategoryAlreadyExists()	2毫秒
✓ CategoryManagementIntegrationTest (com.grooming.pet.controller)	1秒 104毫秒
✓ createCategory_shouldRejectClientRole()	523毫秒
✓ adminCrudFlow_shouldCreateUpdateAndDeleteCategory()	178毫秒
✓ createCategory_shouldRejectBlankName()	153毫秒
✓ deleteCategory_shouldRejectWhenReferencedBySpecialist()	236毫秒
✓ getCategories_shouldBePublicAndReturnPersistedData()	14毫秒

Fig.10. Testing of PBI 4 Expertise Categories Management